IC526 Advanced Compilers

Lec 19: Backend

Yoonseung Kim

DGIST EECS

Fall 2026

Small Patch

bin/main.ml (line numbers in the current file)

Line 138: change the default target

```
- | None -> Llvm_target.Target.default_triple ()
+ | None -> "riscv64-unknown-linux-gnu"
```

Line 145: add target features after TargetMachine.create ~triple

```
144   Llvm_target.TargetMachine.create ~triple
145     ~features:"+m,+a,+f,+d,+c,+zicsr,+zifencei"
146     ~level:(Passes.codegen_level !optimization_level)
```

Running Example: Sum 1 Through n

```
fn sum_to(n: i32) -> i32 {
  let mut sum: i32 = 0;
  let mut i: i32 = 1;
  while i <= n {
    sum = sum + i;
    i = i + 1;
  }
  return sum;
}
```

```
fn main() -> i32 {
  println!("{}", sum_to(10));
  return 0;
}
```

sum_to(10) computes 1 + 2 + … + 10 = 55.

Assembly for Inspection

```
dune exec rustyc -- \
  -S --mem2reg -O0 \
  sum.rs -o sum.s
```

Inspect the instructions produced by the LLVM backend.

LLVM IR: sum_to

```
define i32 @sum_to(i32 %0) {
```

entry:

```
br label %cond
```

cond:

```
%sum.0 = phi i32 [ 0, %entry ],
                 [ %add, %body ]
%i.0 = phi i32 [ 1, %entry ],
               [ %add5, %body ]
%le = icmp sle i32 %i.0, %0
br i1 %le, label %body,
           label %endwhile
```

body

repeat

exit

body:

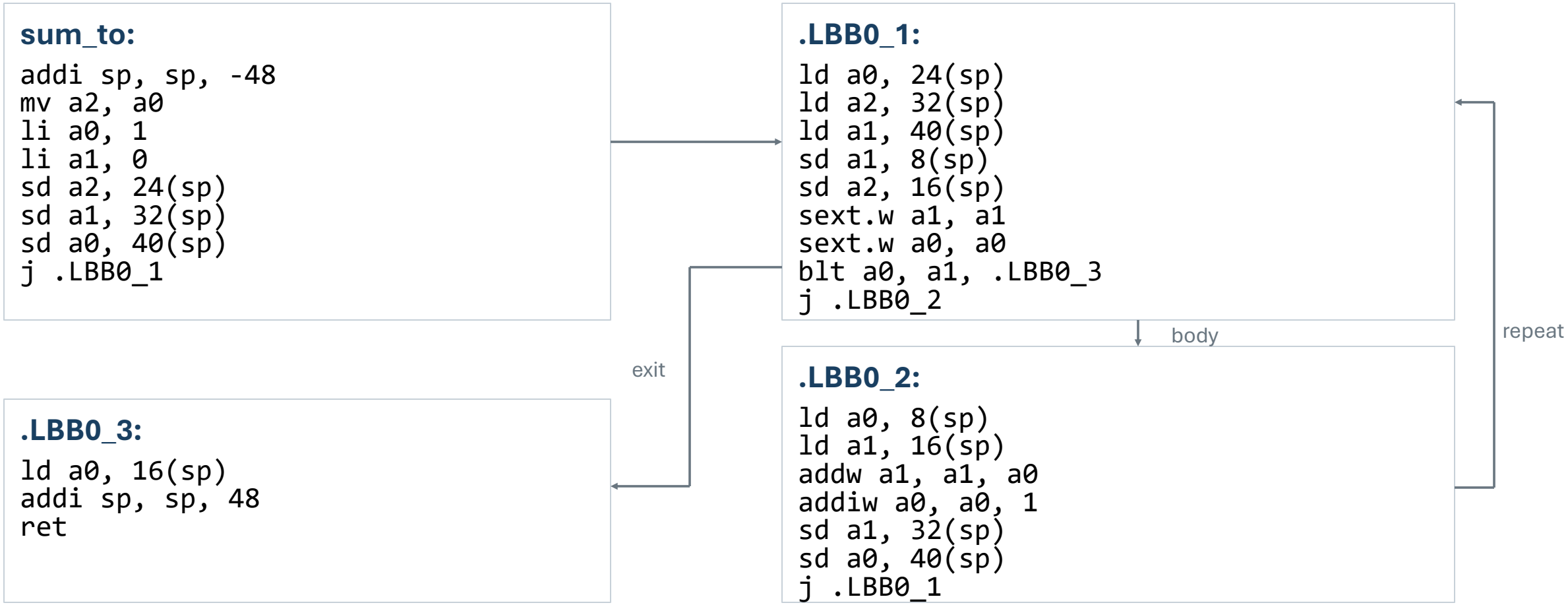
```
%add = add i32 %sum.0, %i.0
%add5 = add i32 %i.0, 1
br label %cond
```

endwhile:

```
ret i32 %sum.0
```

```
}
```

Generated Assembly: sum_to



Generated Assembly: main

main receives control, calls sum_to, prints the result, and returns.

main:

```
addi sp, sp, -16
sd ra, 8(sp)
li a0, 10
call sum_to
mv a1, a0
.Lpcrel_hi0:
auipc a0, %pcrel_hi(.Lfmt)
addi a0, a0, %pcrel_lo(.Lpcrel_hi0)
call printf
li a0, 0
ld ra, 8(sp)
addi sp, sp, 16
ret

.Lfmt: .asciz "%d\n"
```

standard runtime

calls main

receives its exit status

sum_to

input: n in a0

output: sum in a0

printf

a0: format address

a1: integer to print

Object File & Executable

```
dune exec rustyc -- \
  -c --mem2reg -O0 \
  sum.rs -o sum.o

riscv64-linux-gnu-gcc -static sum.o -o sum
qemu-riscv64 ./sum
```

Output: 55

To RISC-V

RISC-V Registers

RV64 has 32 integer registers, x0 to x31. Each register holds 64 bits.

ABI names	Role
`zero`	Always reads as 0
`ra`	Return address written by a call
`sp`	Stack pointer
`a0–a7`	Arguments, with a0/a1 for results. Calls may overwrite them.
`t0–t6`	Temporary values that calls may overwrite
`s0–s11`	A function must restore any of these that it changes
`gp, tp`	Global and thread pointers reserved by the ABI

These names identify physical registers.
We will assign LLVM's virtual registers (e.g., %x) to physical registers.

Calls, Returns, and Stack Frames

A function reserves stack memory by adjusting sp.

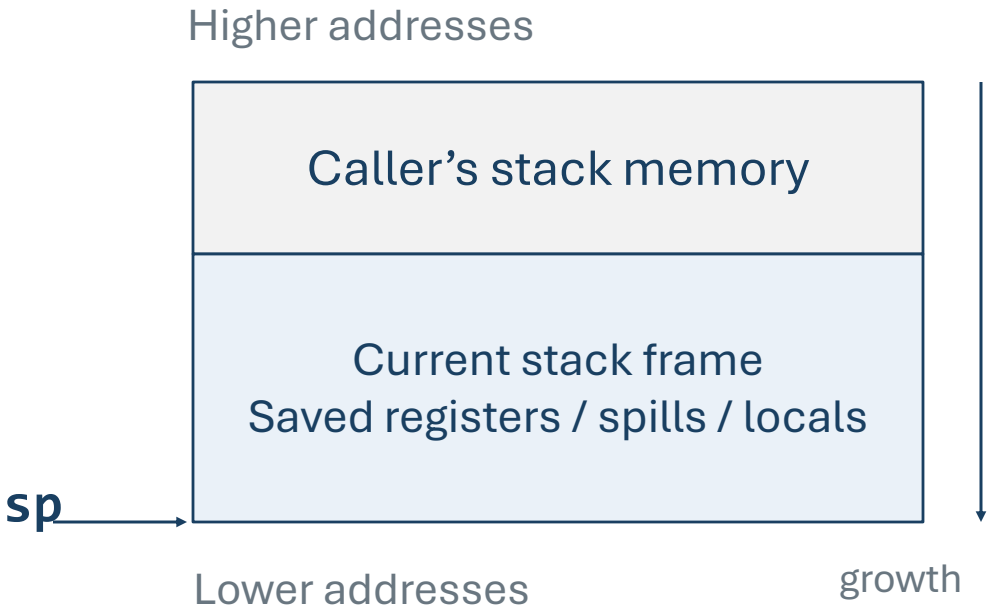
`call sum_to`

Write the return address to ra,
then jump to sum_to.

`ret`

Jump to the address in ra.

A nested call overwrites ra.
Save it if it is still needed.




```
Reserve: sp = sp - frame_size
Release: sp = sp + frame_size
```

What Each Stage Decides

Stage	Question answered
Instruction selection	Which machine operations implement the IR?
ϕ elimination	How does each incoming edge deliver its values?
Register allocation	Where do the live values reside?
Calls and stack frames	How do functions exchange and preserve values?
Object files and linking	How do instructions and symbols form an executable?

Instruction Selection

Arithmetic: LLVM IR

```
define i32 @sum_to(i32 %0) {
```

entry:

```
br label %cond
```

cond:

```
%sum.0 = phi i32 [ 0, %entry ],
                 [ %add, %body ]
%i.0 = phi i32 [ 1, %entry ],
               [ %add5, %body ]
%le = icmp sle i32 %i.0, %0
br i1 %le, label %body,
           label %endwhile
```

body

repeat

exit

body:

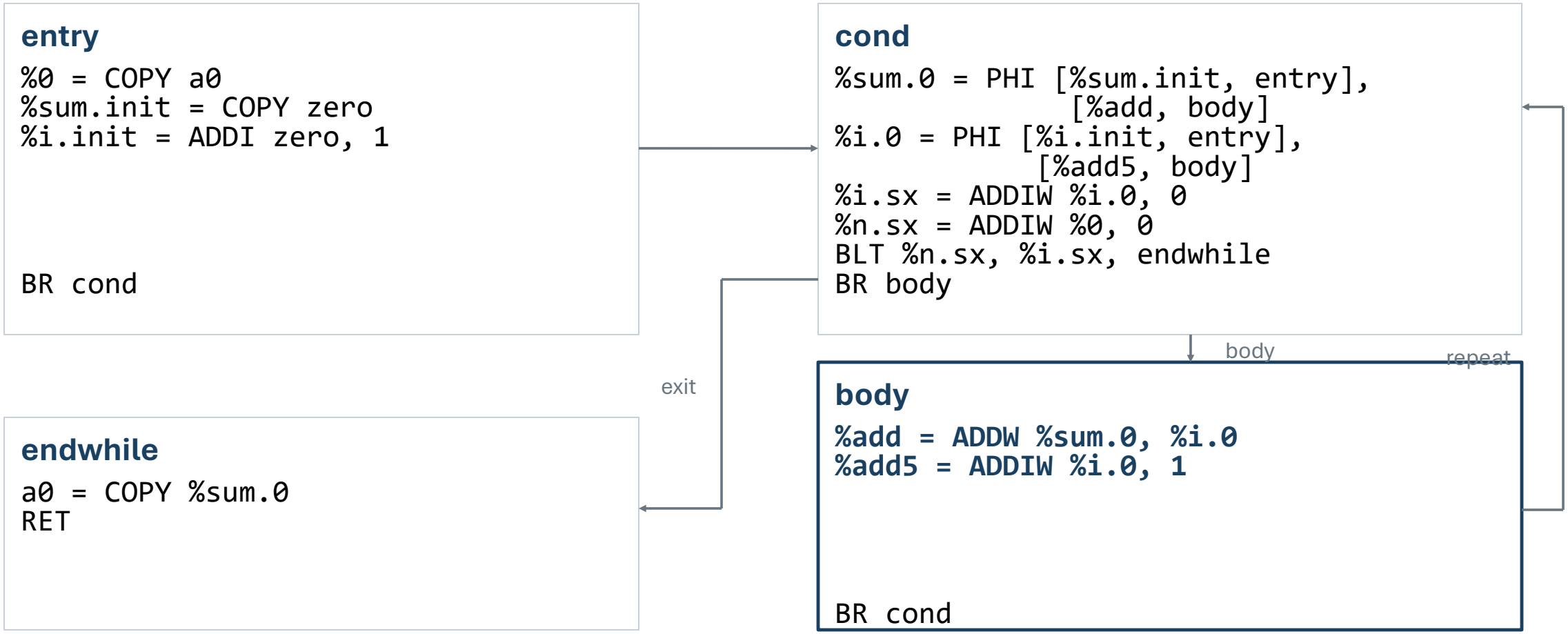
```
%add = add i32 %sum.0, %i.0
%add5 = add i32 %i.0, 1
br label %cond
```

endwhile:

```
ret i32 %sum.0
```

```
}
```

Arithmetic: Machine Instructions



ADDW adds two registers.
ADDIW encodes the constant 1 as an immediate.

Comparison and Branch: LLVM IR

```
define i32 @sum_to(i32 %0) {
```

entry:

```
br label %cond
```

cond:

```
%sum.0 = phi i32 [ 0, %entry ],
                 [ %add, %body ]
%i.0 = phi i32 [ 1, %entry ],
               [ %add5, %body ]
%le = icmp sle i32 %i.0, %0
br i1 %le, label %body,
           label %endwhile
```

body

repeat

exit

body:

```
%add = add i32 %sum.0, %i.0
%add5 = add i32 %i.0, 1
br label %cond
```

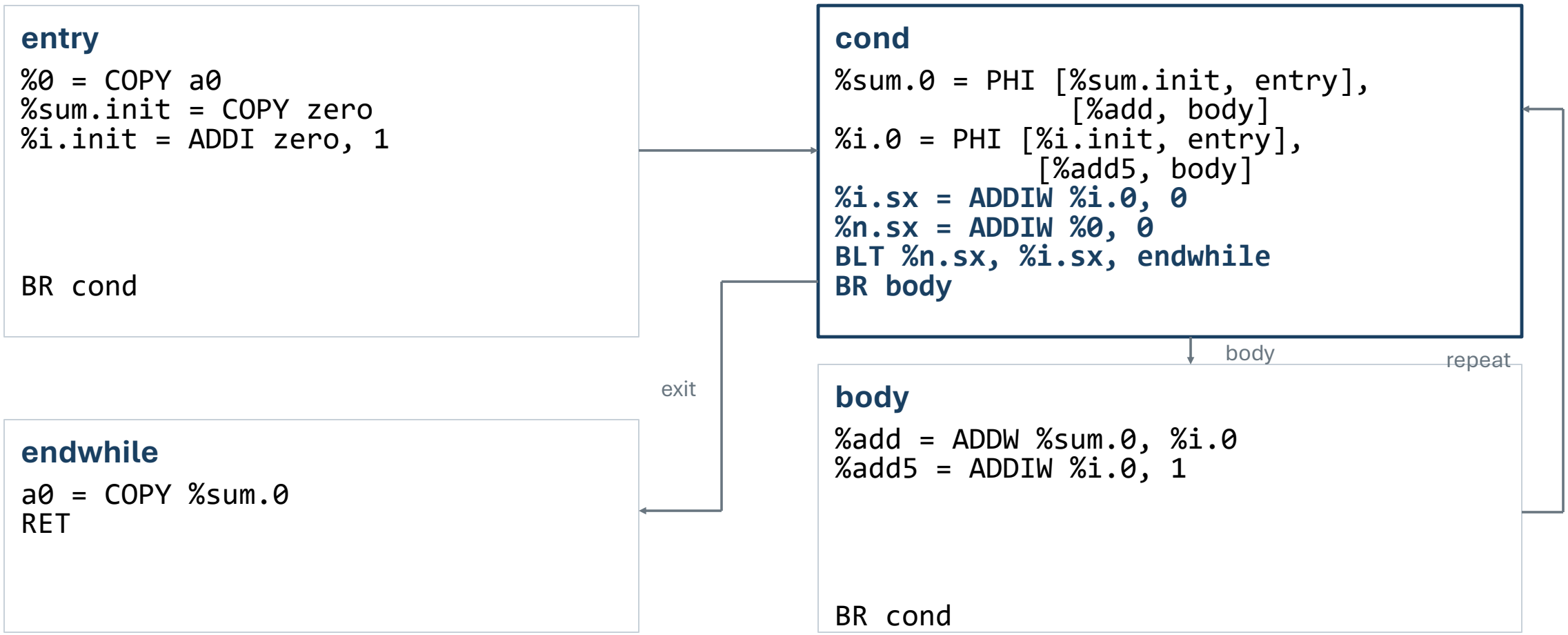
endwhile:

```
ret i32 %sum.0
```

```
}
```

Comparison and Branch: Machine IR

Sign-extend the operands, then branch to endwhile when %0 < %i.0



Initial Constants: LLVM IR

```
define i32 @sum_to(i32 %0) {
```

entry:

```
br label %cond
```

cond:

```
%sum.0 = phi i32 [ 0, %entry ],
                 [ %add, %body ]
%i.0 = phi i32 [ 1, %entry ],
               [ %add5, %body ]
%le = icmp sle i32 %i.0, %0
br i1 %le, label %body,
           label %endwhile
```

body

repeat

exit

body:

```
%add = add i32 %sum.0, %i.0
%add5 = add i32 %i.0, 1
br label %cond
```

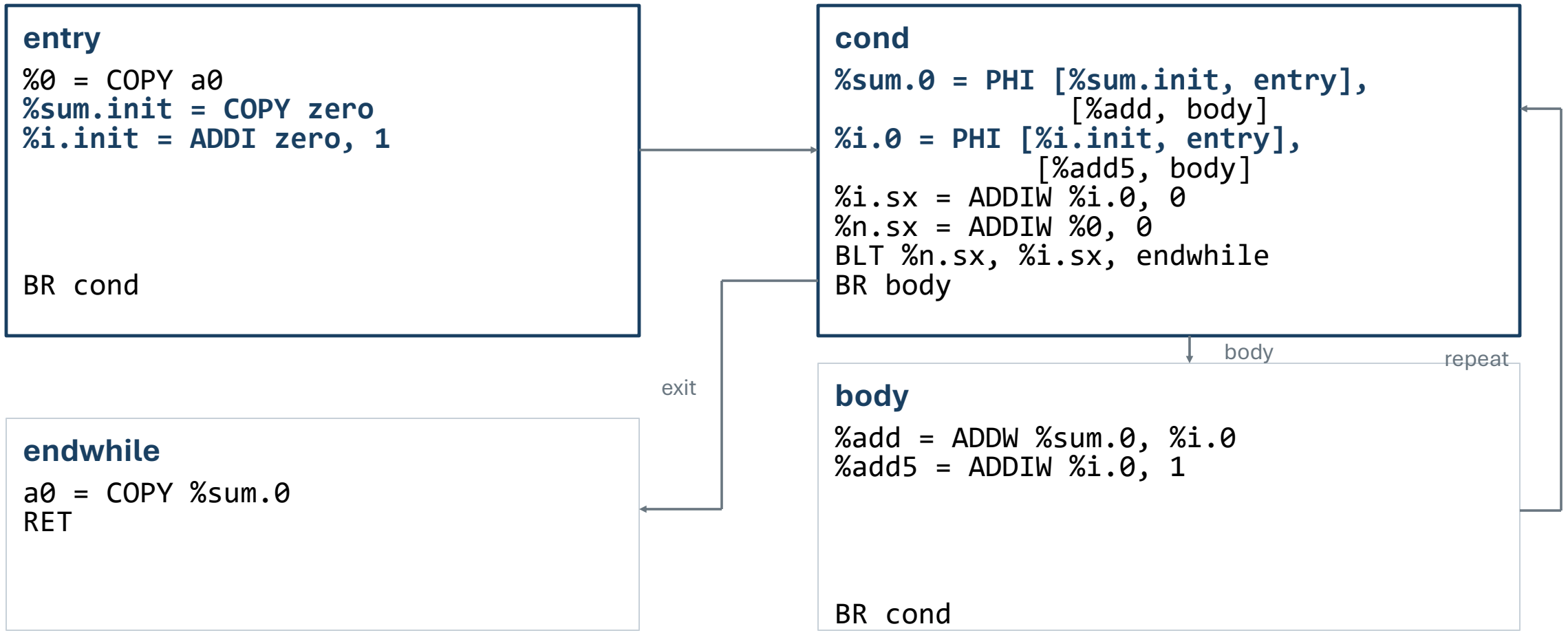
endwhile:

```
ret i32 %sum.0
```

```
}
```

Initial Constants: Machine Instructions

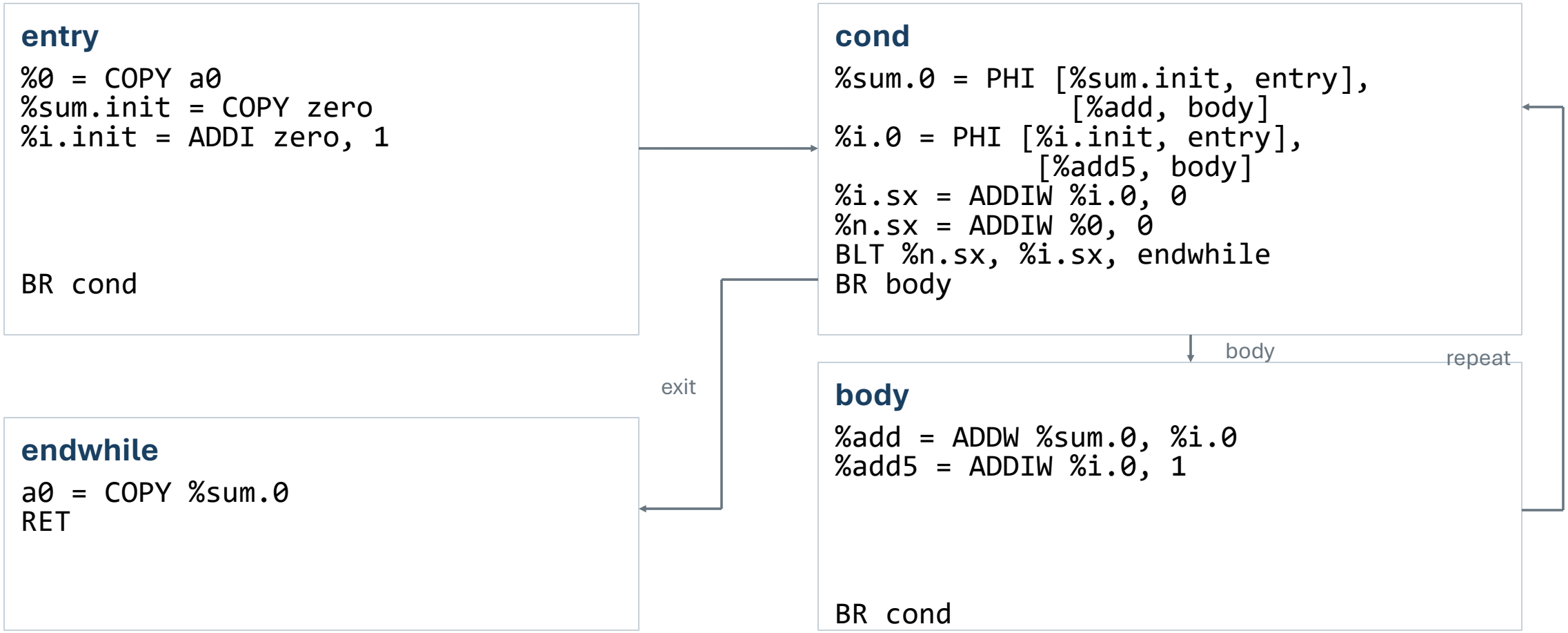
Machine PHIs take registers: entry prepares the values 0 and 1.



zero = x0, which always stores 0

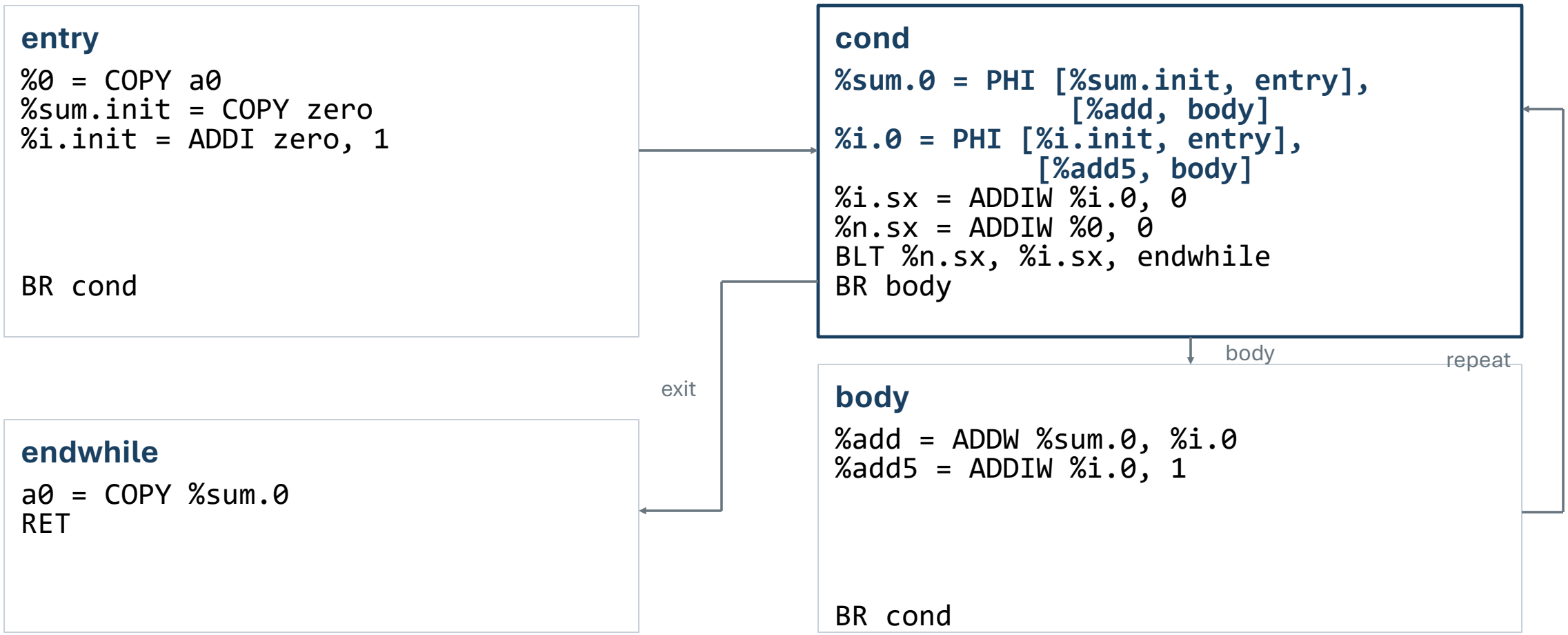
Selected Machine Instructions

RISC-V machine operations, plus compiler-internal PHI and COPY instructions.



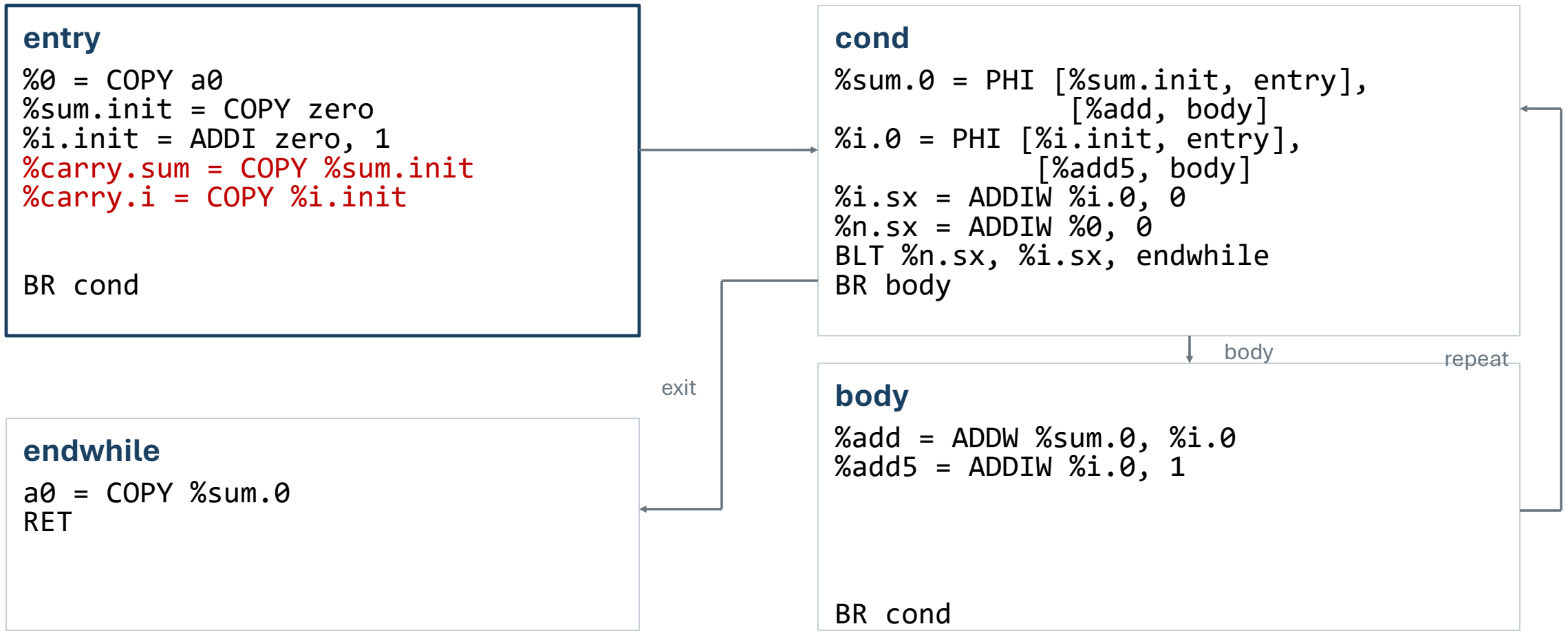
ϕ Elimination

What Must Reach the Loop Header?



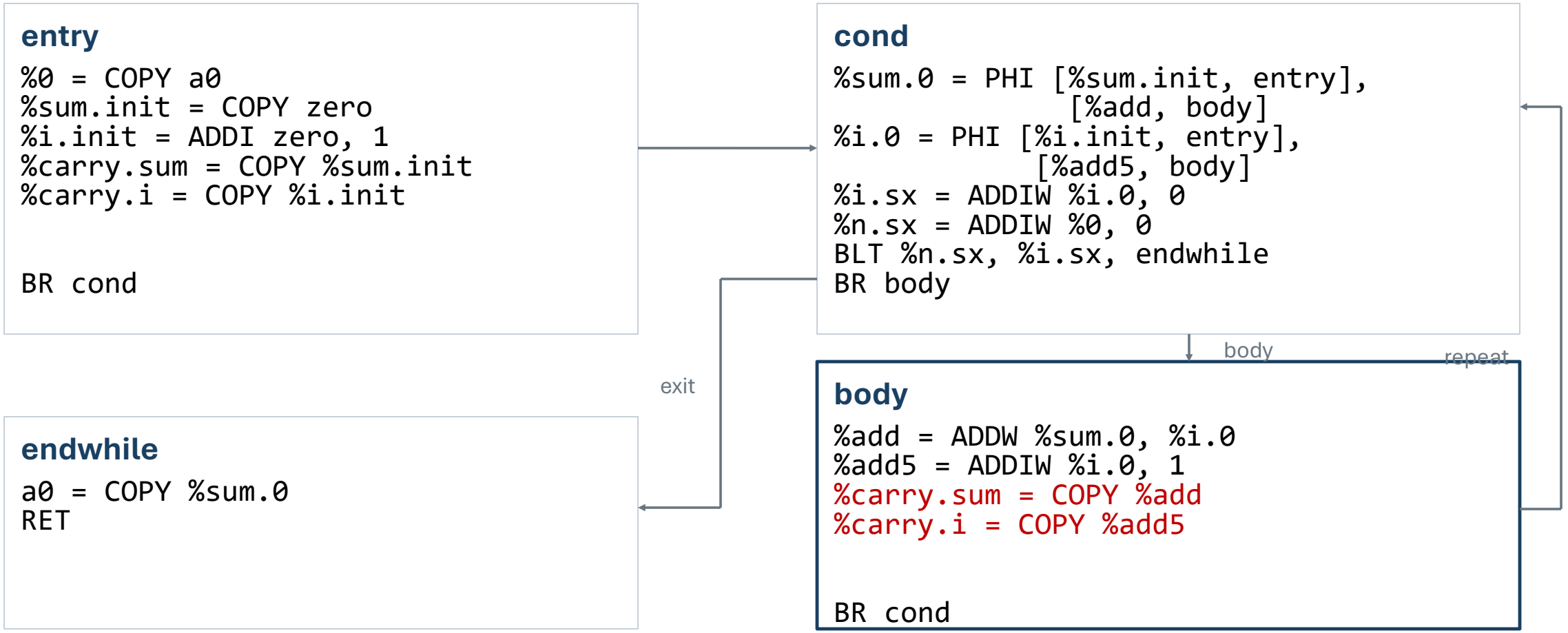
Prepare the Entry Values

Before entry reaches cond, copy the initial values into shared virtual registers.



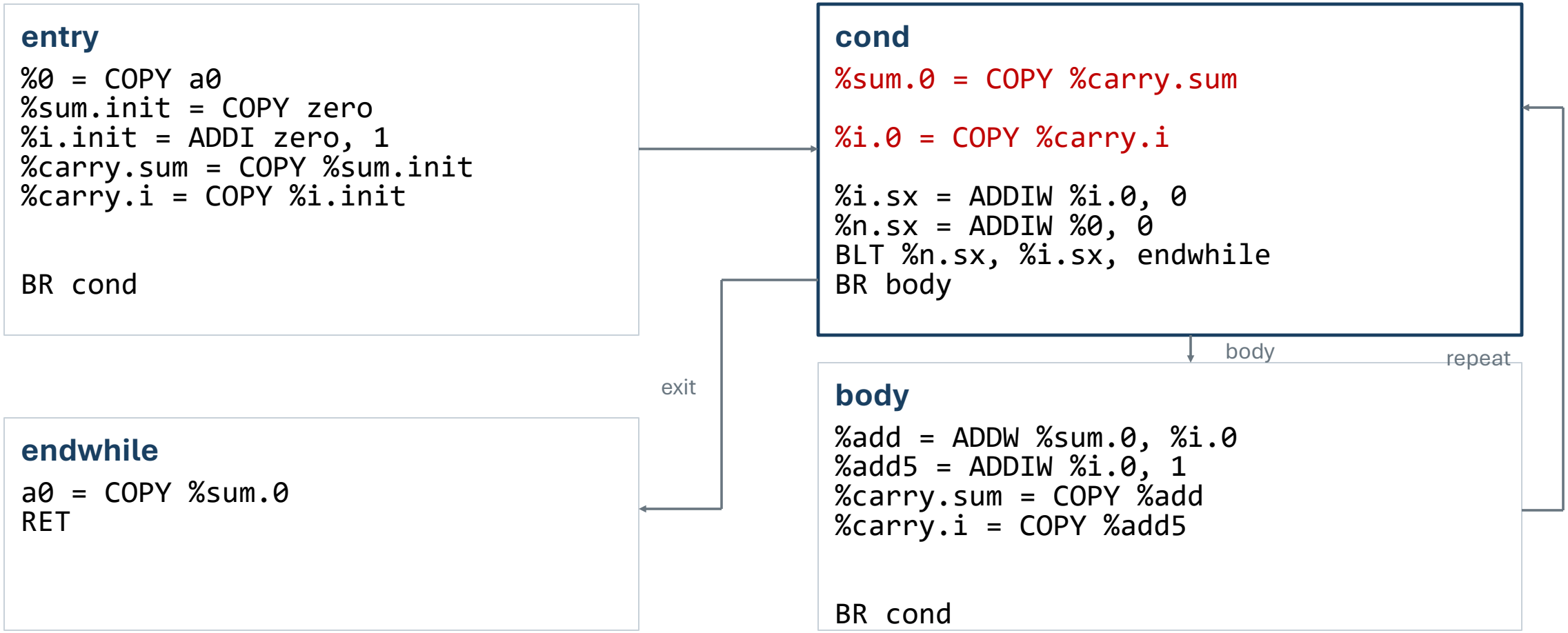
Prepare the Backedge Values

Before body returns to cond, copy the next values into the same registers.



Replace ϕ with Ordinary Copies

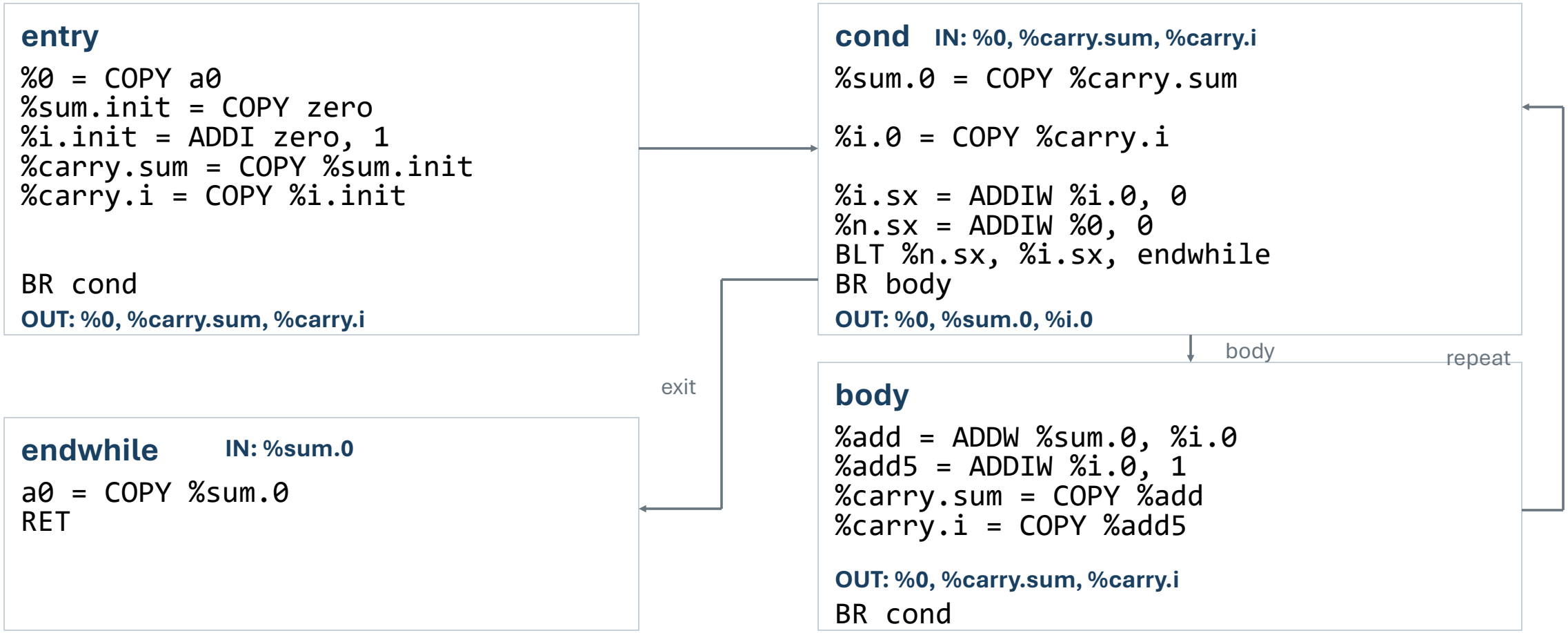
The path already prepared the values. The header can read them directly.



Register Allocation

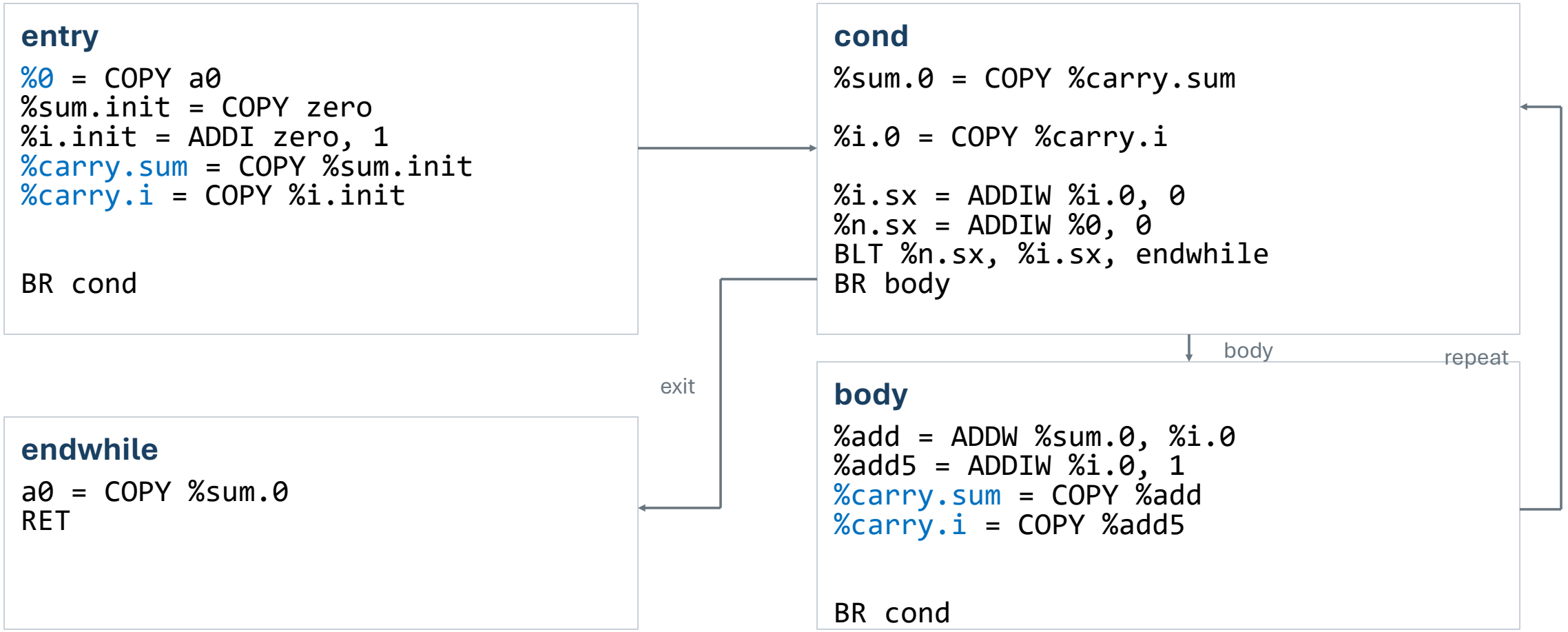
Liveness Across the Whole CFG

A value is live if a future path will use it. Include the loop backedge.



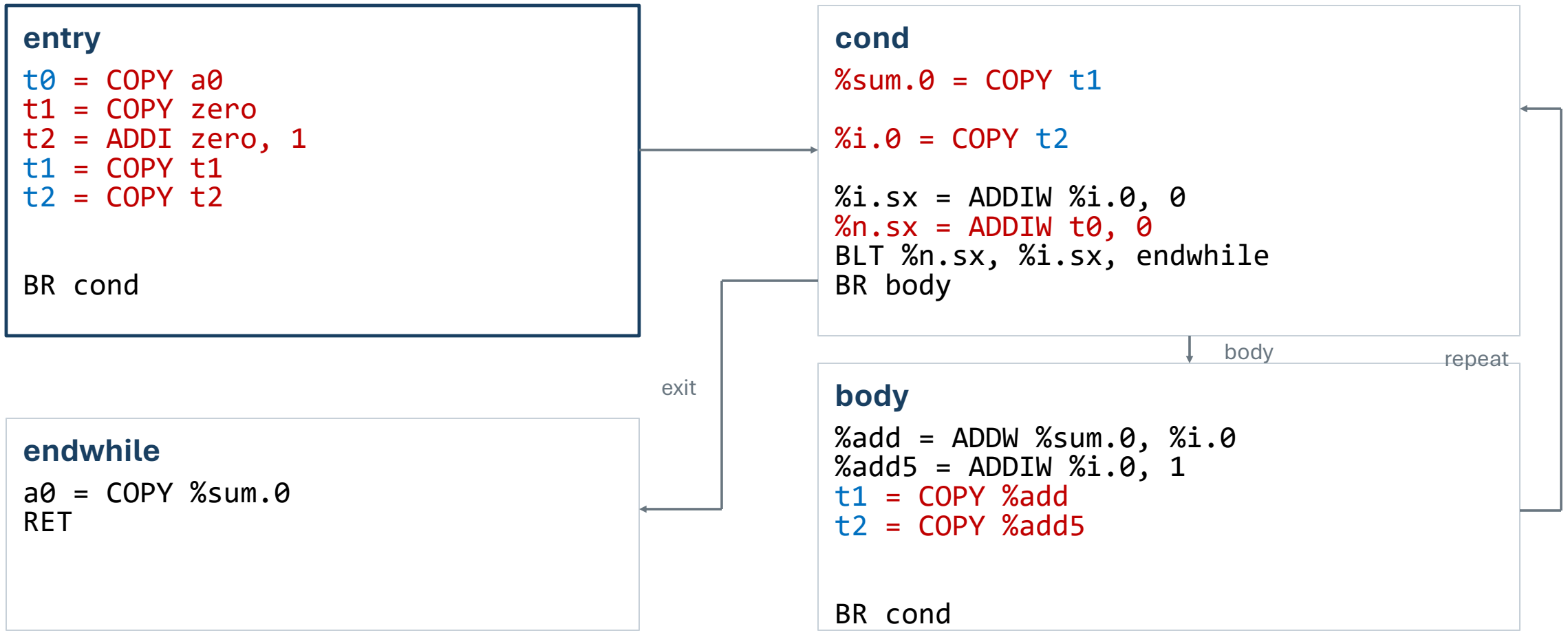
A Simple Allocation Strategy

Use the precomputed liveness to choose registers; release one after its last use.



Entry: Assign the Initial Values

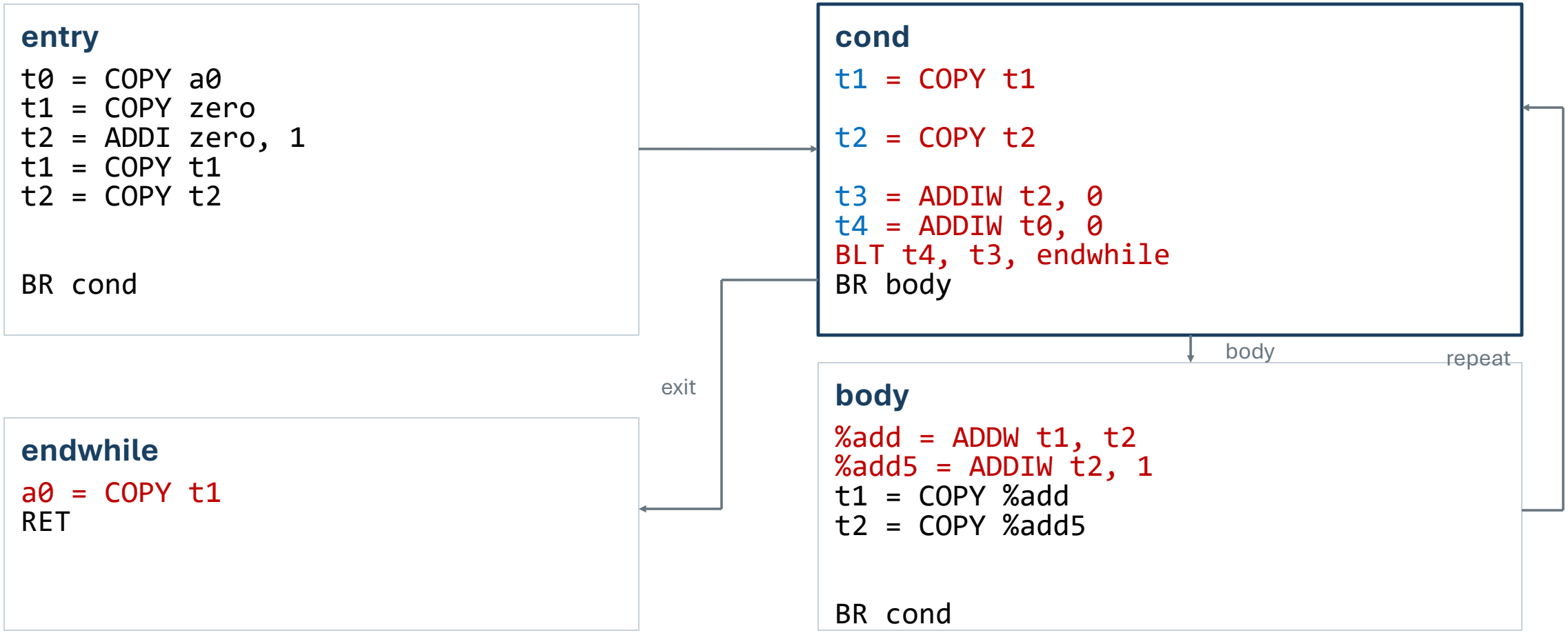
%0 uses t0; initial and carried sum use t1; initial and carried i use t2.



The three values are live together on entry to cond, so use distinct registers.

Cond: Assign the Incoming Values

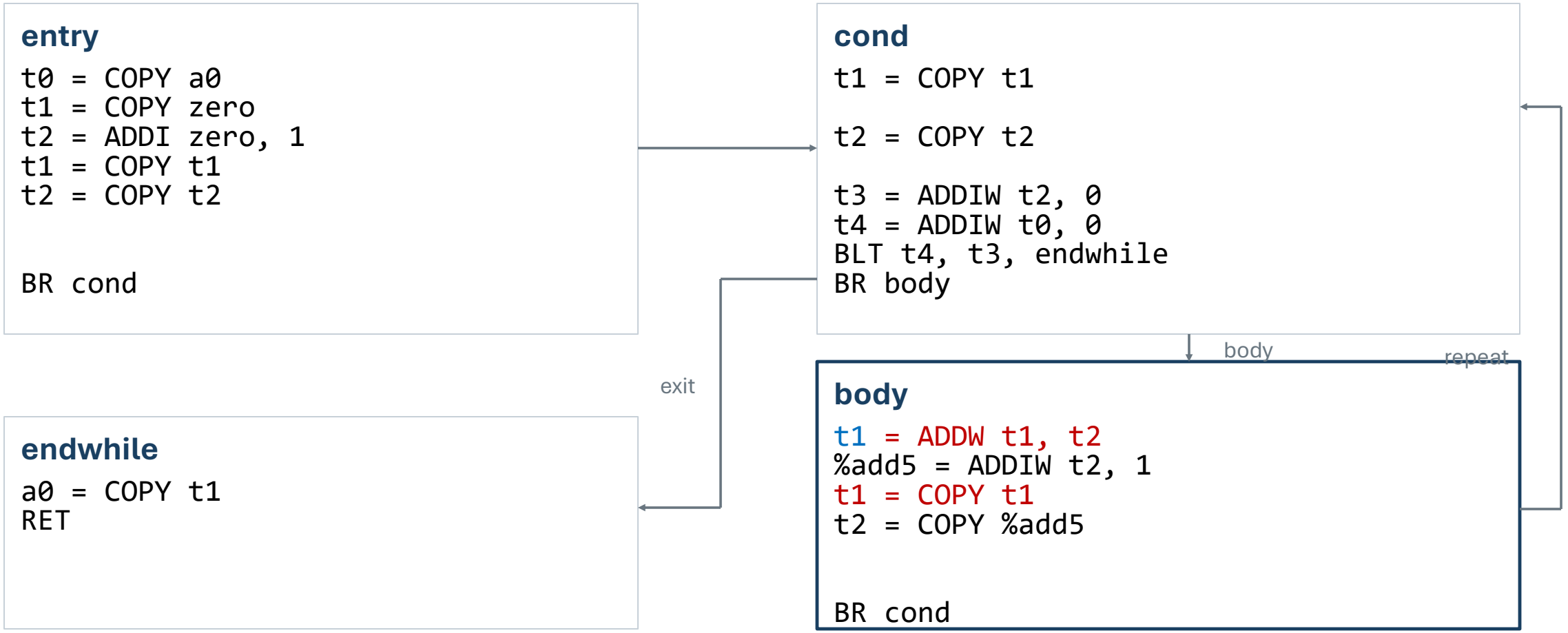
%sum.0 uses t1, %i.0 uses t2; the sign-extended operands use t3 and t4.



t3 and t4 die at BLT. Along the body edge, t0, t1, and t2 remain live.

Body: Reuse the Sum Register

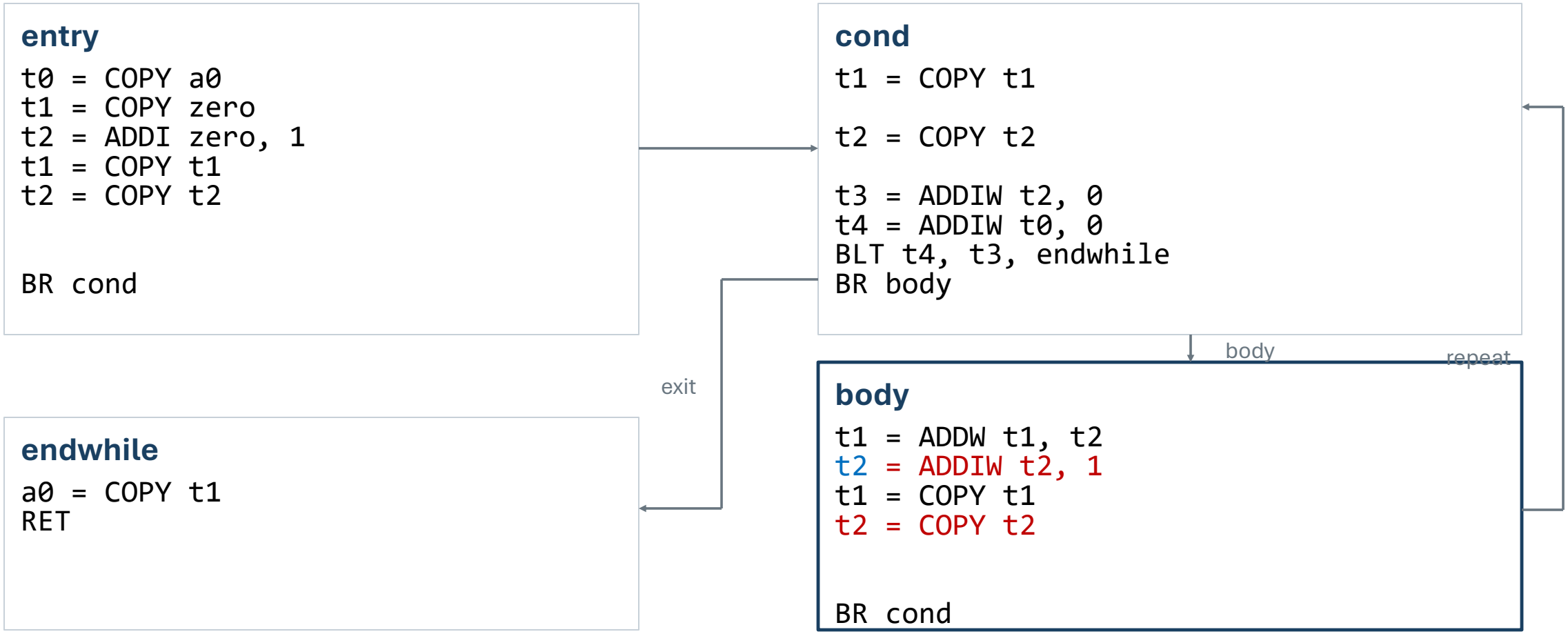
ADDW is the last use of the old sum, so assign %add to t1.



ADDW reads t1 and t2 before writing t1. t2 still holds the old i.

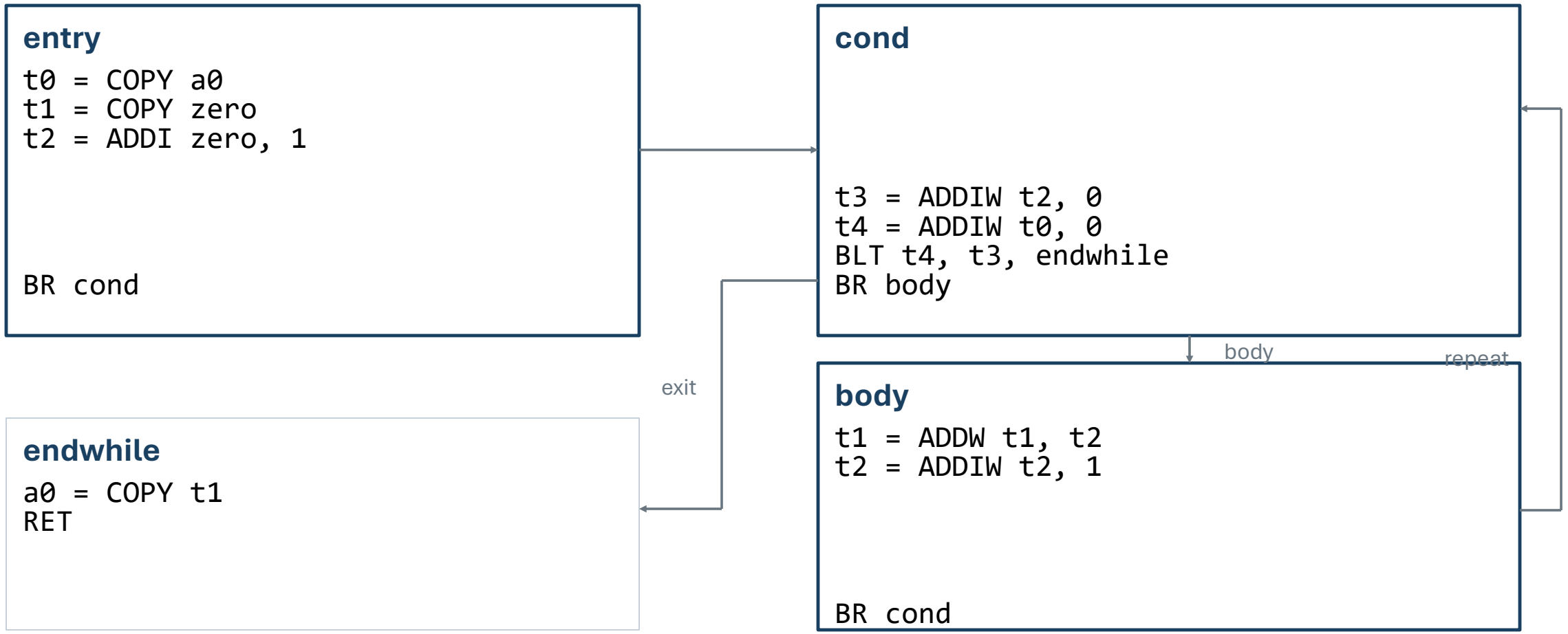
Body: Reuse the i Register

ADDIW is the last use of the old i, so assign %add5 to t2.



Backedge: The Next Values Are Ready

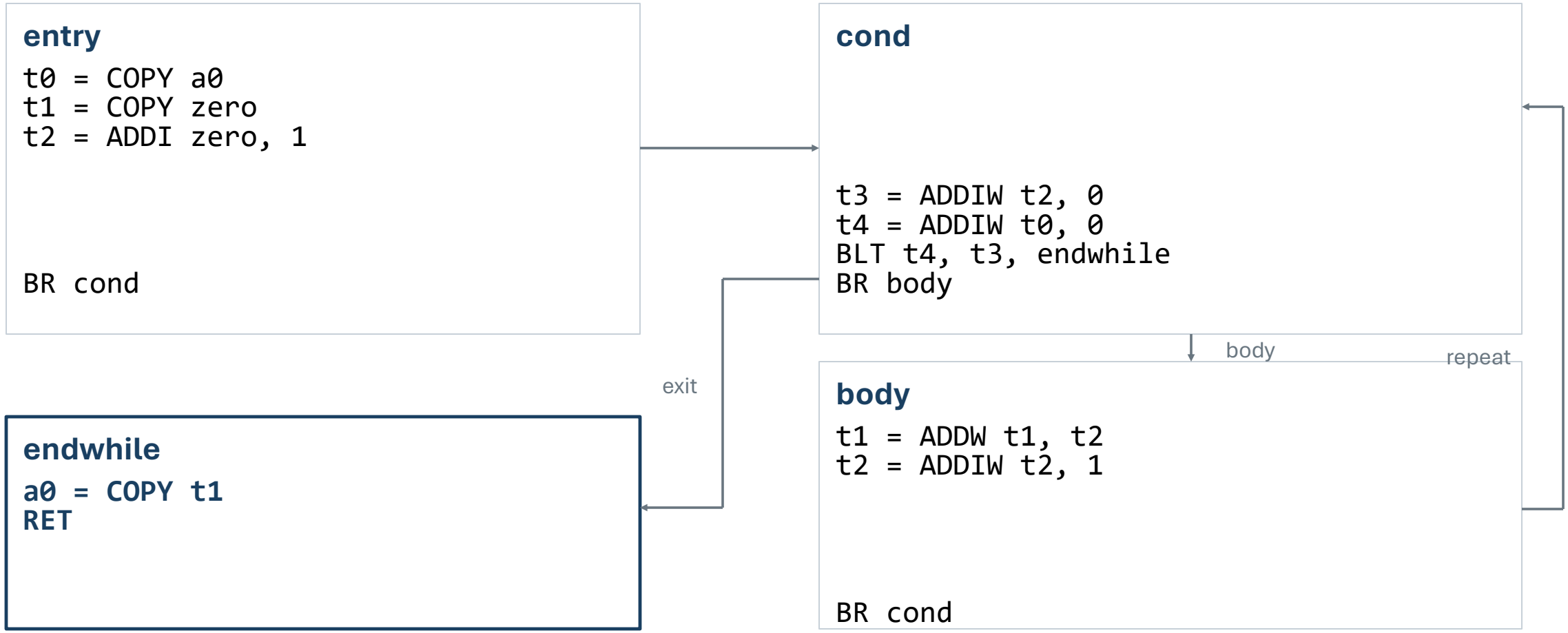
The next iteration already receives the right values in t0, t1, and t2.



All six carry-related copies are now COPY r, r. Remove them across the CFG.

Endwhile: Return the Sum in a0

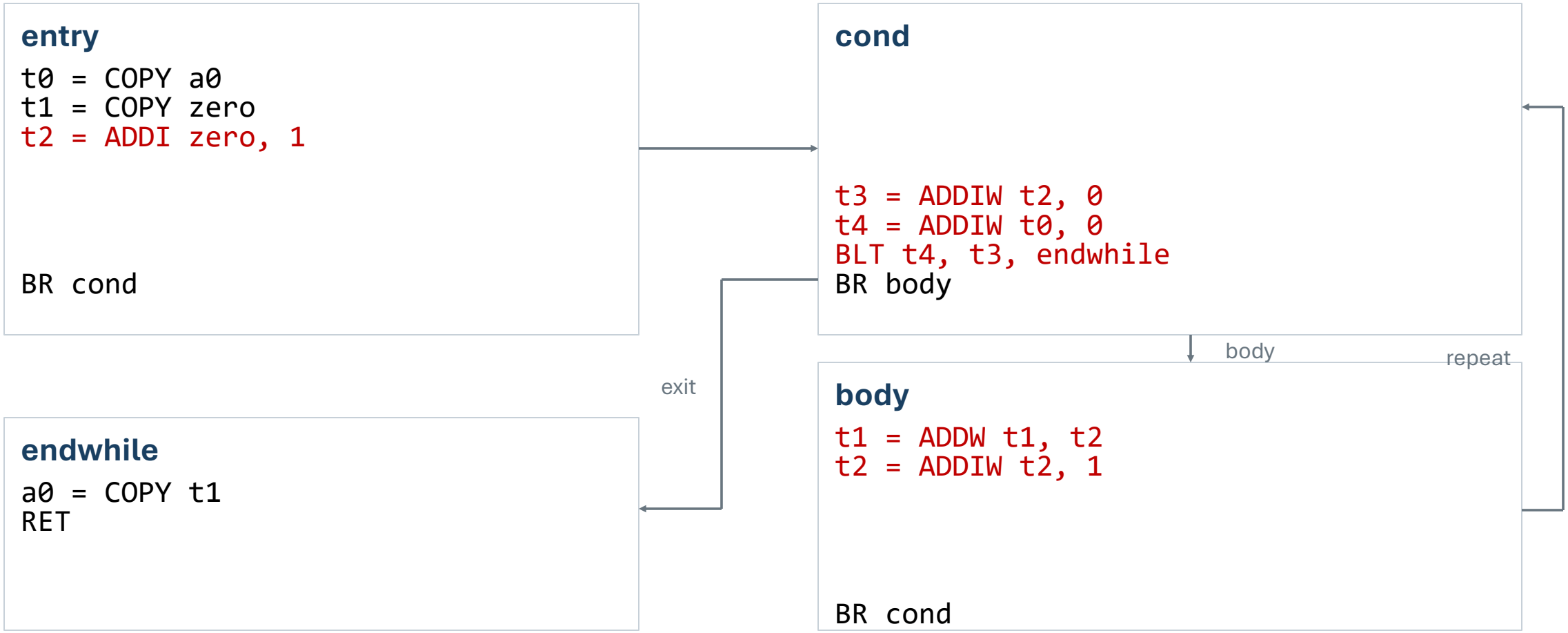
On the exit edge, only the sum in t1 is needed as the return value.



The calling convention fixes the result in a0. RET jumps to the address in ra.

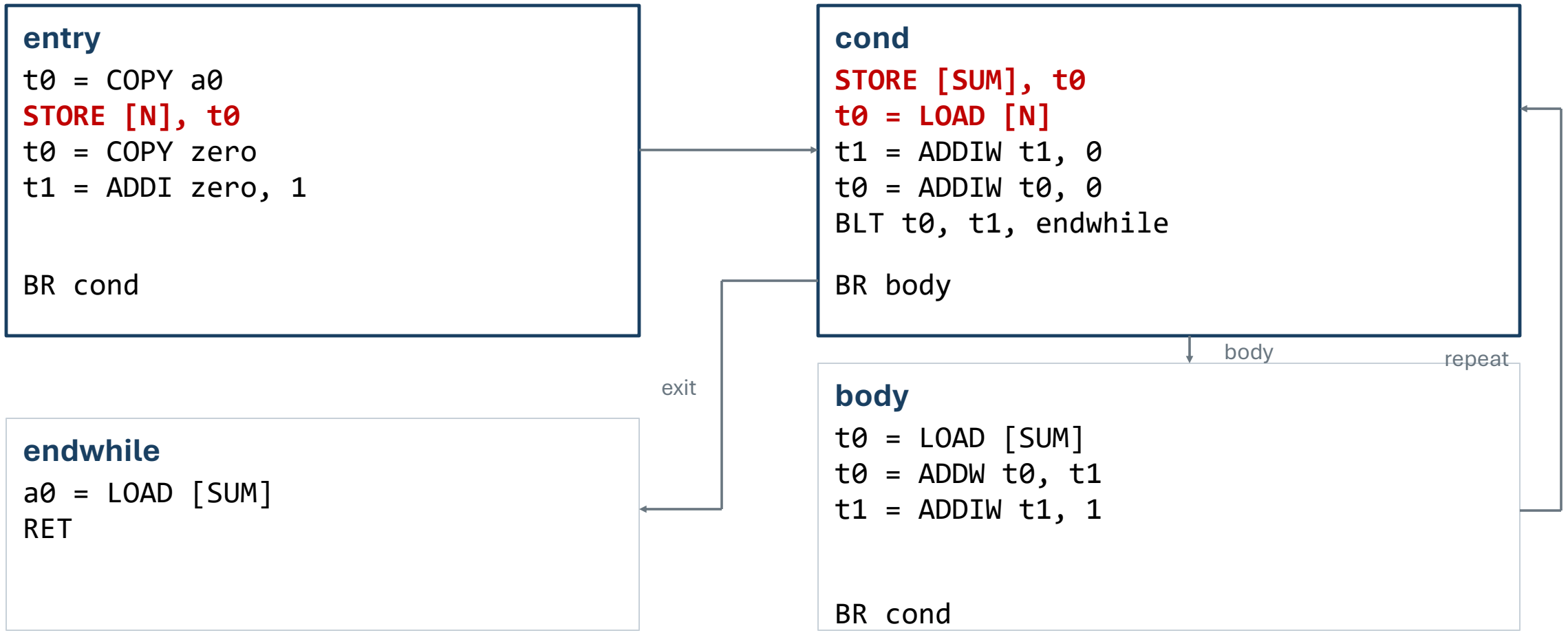
Spilling: A Two-register Budget

Assume only t0 and t1 are available for allocation. The previous result needs more.



Spilling: Save a Value Before Reuse

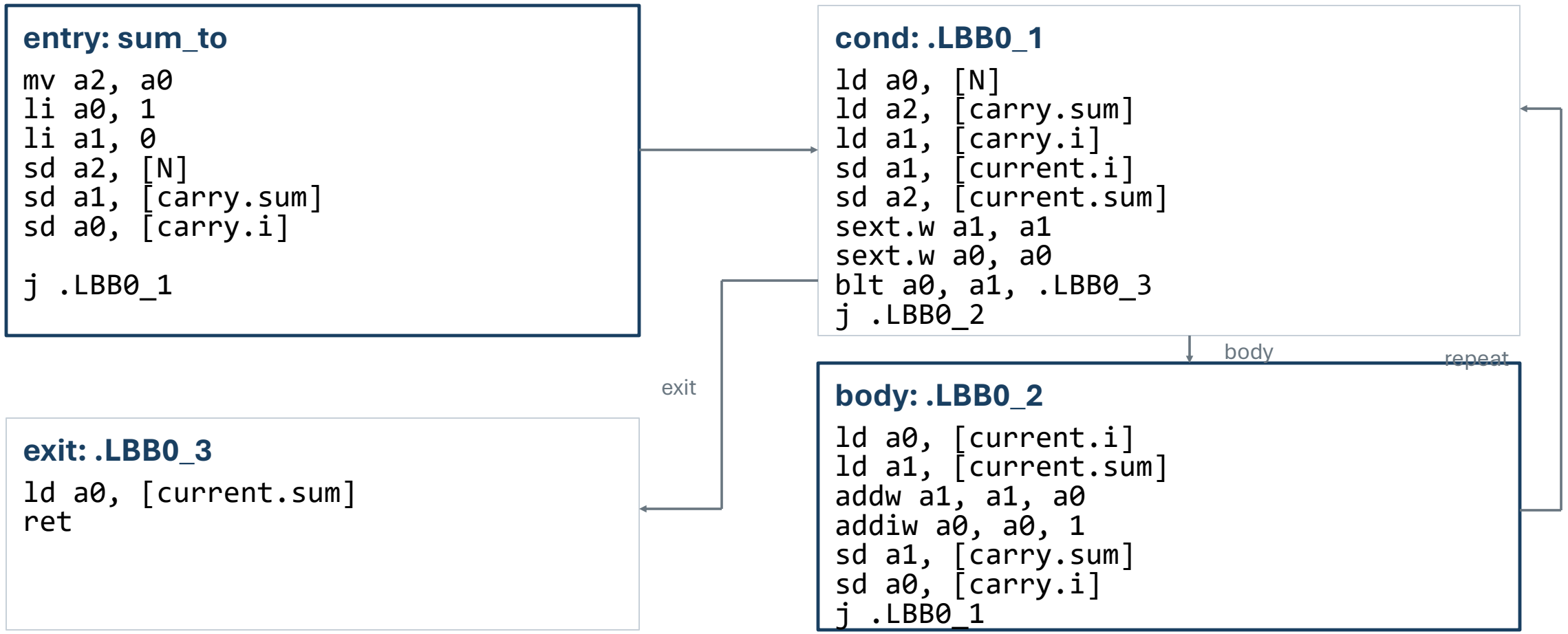
We share t0 for N and SUM, using the stack space.



[N], [SUM] are memory addresses, which are not determined yet.

Actual -O0: Spills Between Blocks

LLVM -O0 uses FastRA, which assigns registers one basic block at a time.



Calls and Stack Frames

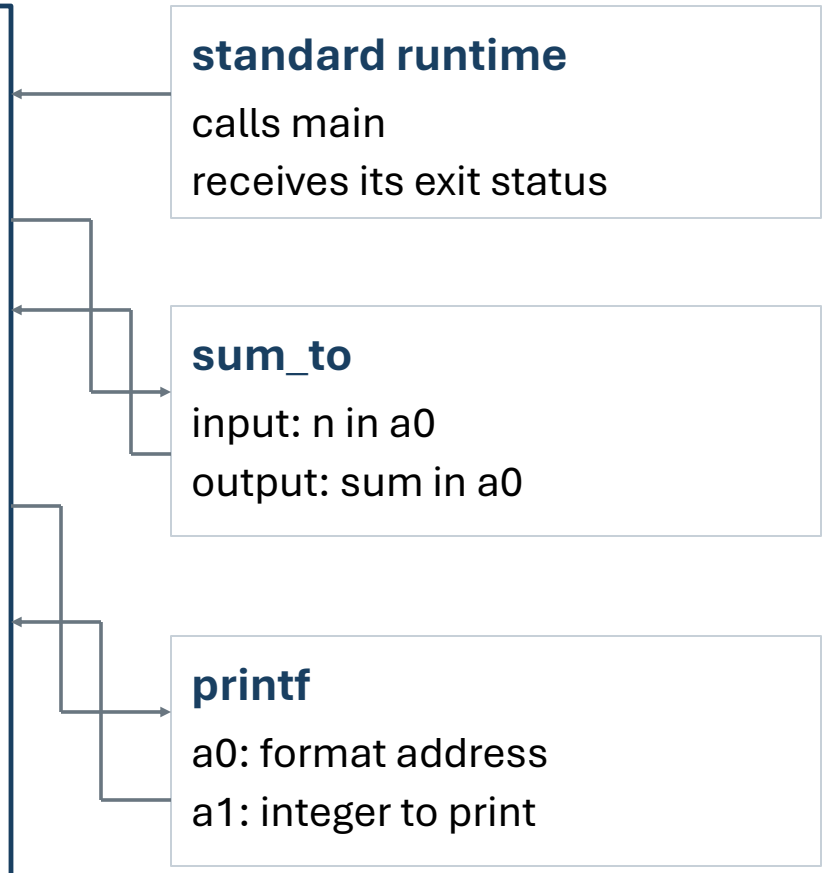
Arguments and Results Use Fixed Registers

a0 holds the first argument or result; a1 holds the second argument.

main

```
%saved.ra = COPY ra
li a0, 10
call sum_to
mv a1, a0
.Lpcrel_hi0:
auipc a0, %pcrel_hi(.Lfmt)
addi a0, a0, %pcrel_lo(.Lpcrel_hi0)
call printf
li a0, 0
ra = COPY %saved.ra
ret

.Lfmt: .asciz "%d\n"
```



Move the sum into a1 before replacing a0 with the format-string address.

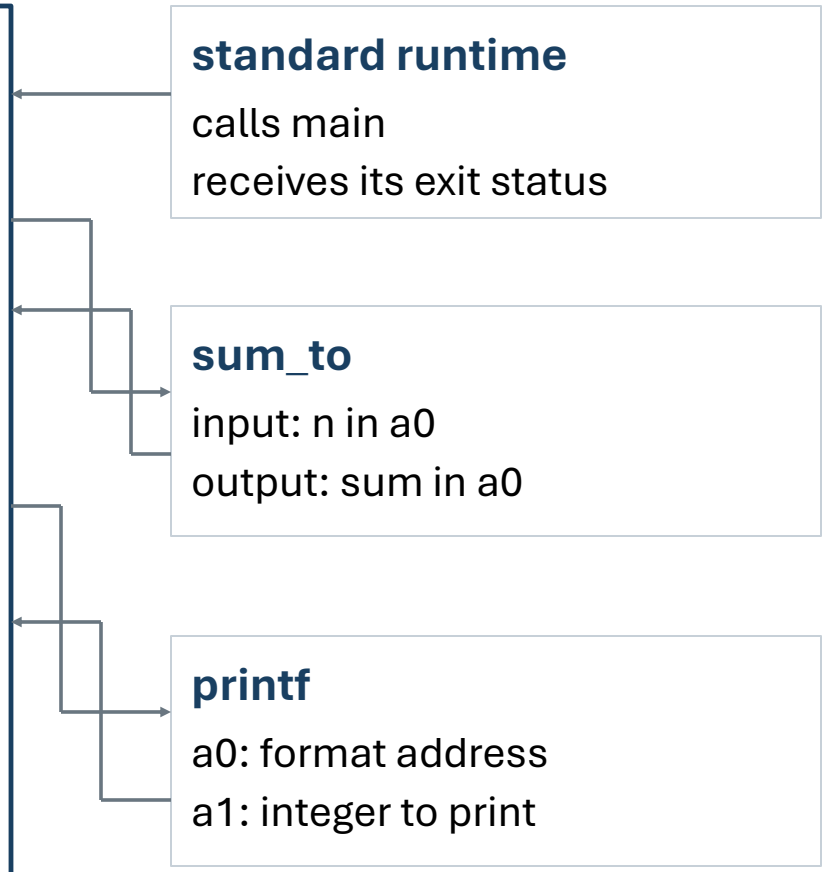
A Nested Call Overwrites ra

Each call writes ra. main must preserve the return address of its own caller.

main

```
%saved.ra = COPY ra
li a0, 10
call sum_to
mv a1, a0
.Lpcrel_hi0:
auipc a0, %pcrel_hi(.Lfmt)
addi a0, a0, %pcrel_lo(.Lpcrel_hi0)
call printf
li a0, 0
ra = COPY %saved.ra
ret

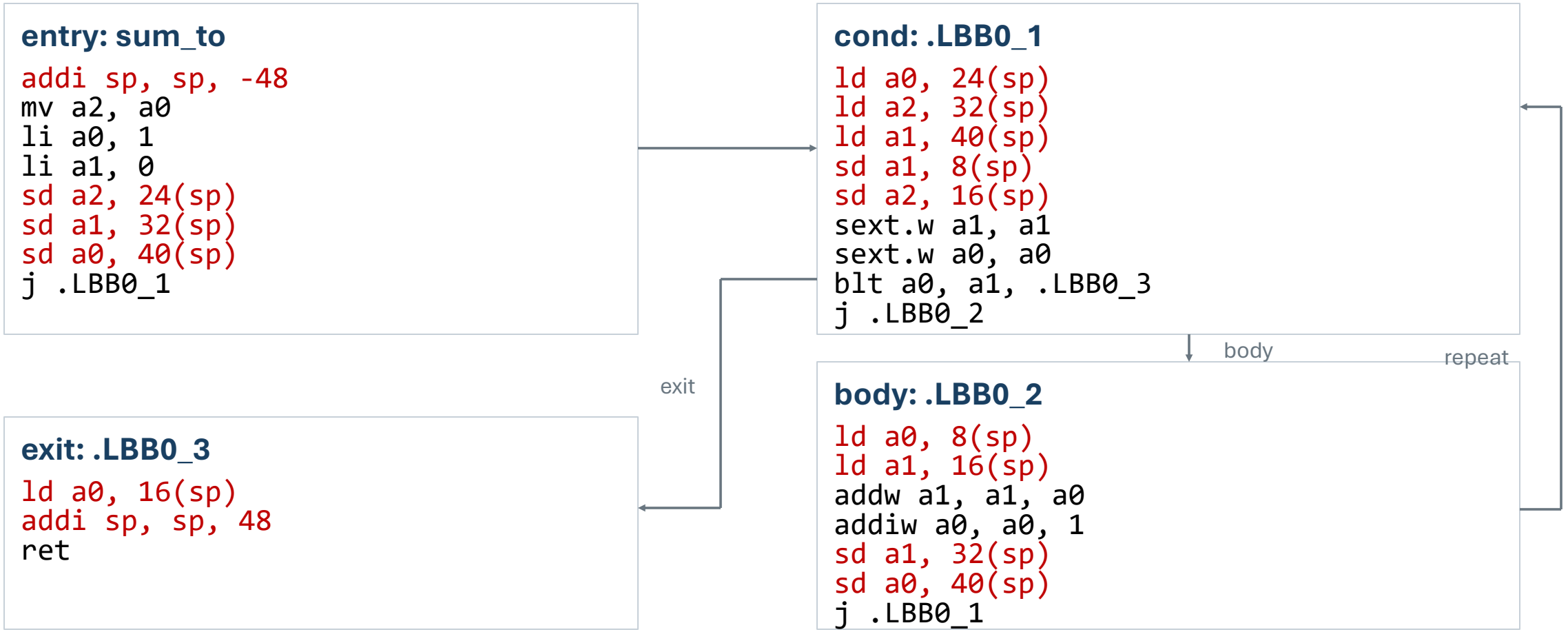
.Lfmt: .asciz "%d\n"
```



%saved.ra must survive both calls. Restore it before returning to the runtime.

Give the Slots Stack Addresses

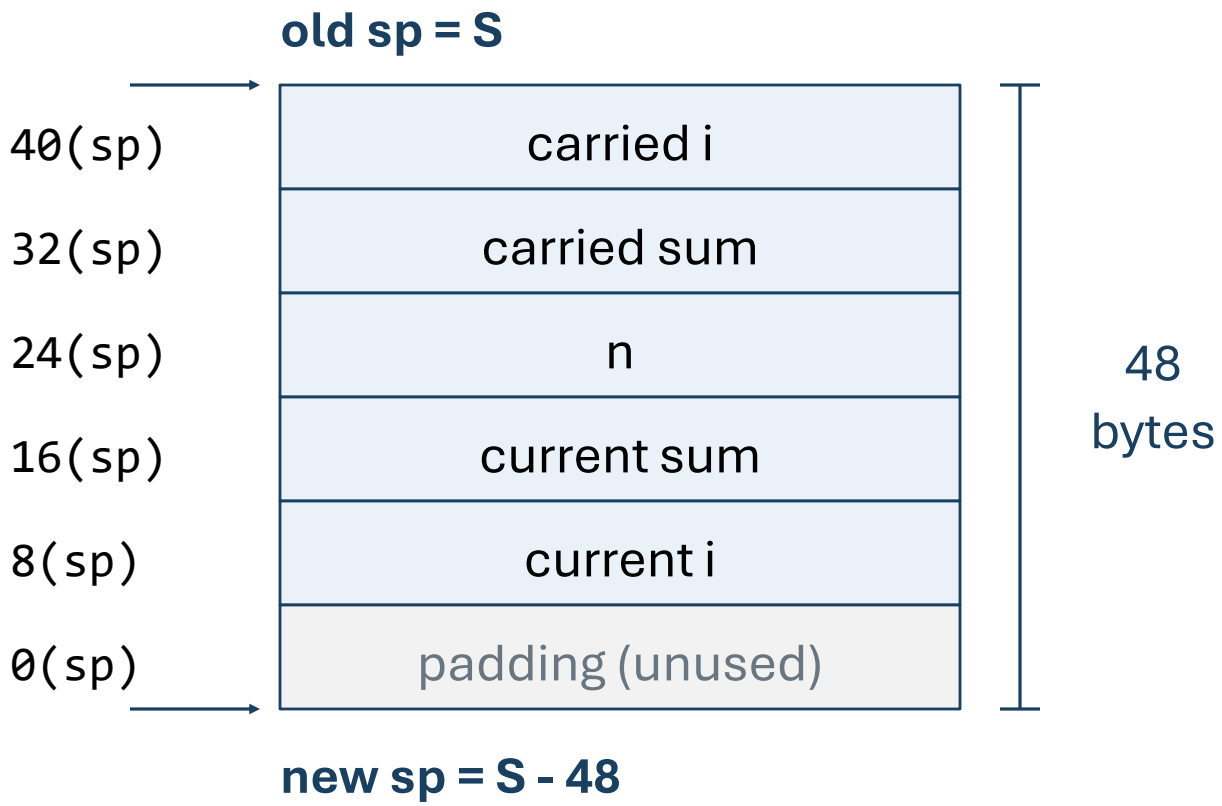
Five 8-byte slots need 40 bytes; 16-byte alignment makes the frame 48 bytes.



Offsets from sp: N=24, carry.sum=32, carry.i=40, current.sum=16, current.i=8.

sum_to: A 48-byte Stack Frame

Each row is 8 bytes. Higher addresses are at the top.



Five spill slots:

5 × 8 = 40 bytes

Keep sp 16-byte aligned:

round 40 up to 48 bytes.

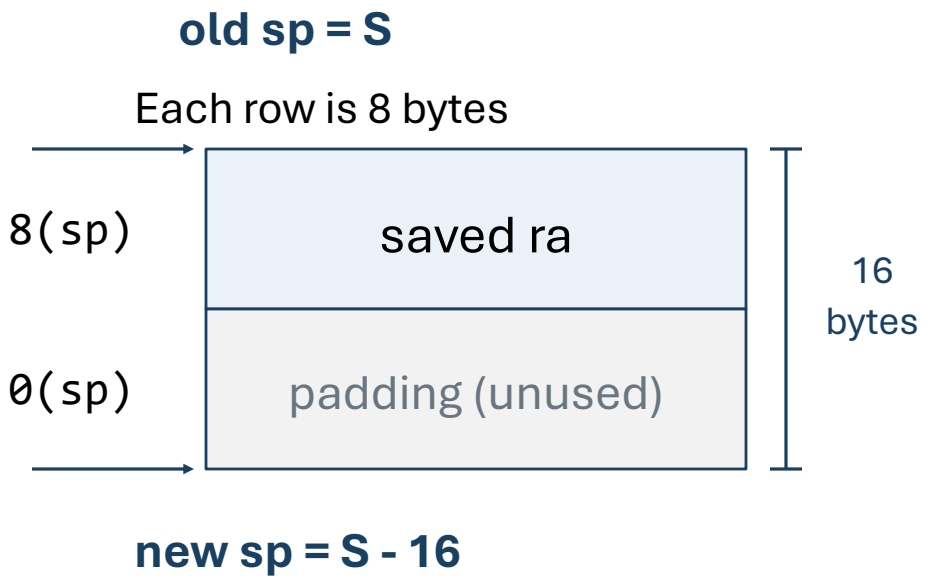
Preserve ra in main's Stack Frame

One 8-byte saved ra plus padding makes a 16-byte frame.

main

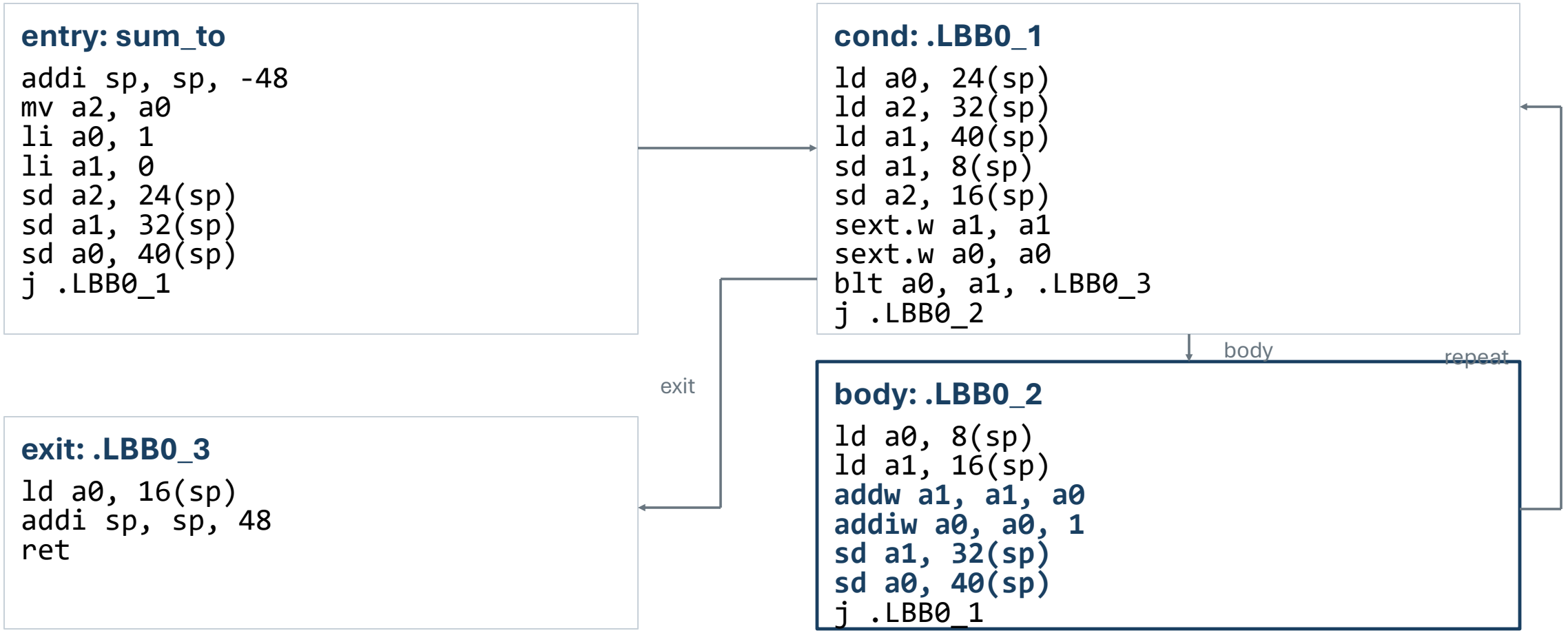
```
addi sp, sp, -16
sd ra, 8(sp)
li a0, 10
call sum_to
mv a1, a0
.Lpcrel_hi0:
  auipc a0, %pcrel_hi(.Lfmt)
  addi a0, a0, %pcrel_lo(.Lpcrel_hi0)
call printf
li a0, 0
ld ra, 8(sp)
addi sp, sp, 16
ret

.Lfmt: .asciz "%d\n"
```



Read the Actual Loop Again

Entry initializes; cond tests; body updates; exit returns the current sum.



The same update and value-transfer principles explain the actual -O0 instructions.

Object Files and Linking

The linker combines object code with startup code and libraries.




```
riscv64-linux-gnu-gcc -static sum.o -o sum
```

GCC is the driver: it invokes the linker with the required runtime files.